

1. BÁO CÁO TỔNG THỂ CÁC MÔ HÌNH NHẬN DIỆN TẾ NGÃ (v30 - v31)

Mục tiêu: Tổng hợp, phân tích và so sánh các kiến trúc mạng Neural Network từ phiên bản v30 đến v31 được tối ưu hóa (TinyML) để chạy trên vi điều khiển ESP32-S3 sử dụng bộ gia tốc ESP-NN.

1.1. 1. DỮ LIỆU VÀ TIỀN XỬ LÝ (DATA PIPELINE)

Tất cả các phiên bản đều sử dụng chung một cơ sở dữ liệu SisFall với cấu hình Pipeline được thiết kế vô cùng tinh xảo: - **Phân loại 5 nhãn (Classes):** Walk, Run, Idle, Trans, Fall (Tế ngã là index 4).

1.1.1. 1.1. Tiền xử lý & Giảm tần số lấy mẫu (Downsampling)

- Tần số lấy mẫu gốc của cảm biến SisFall là 200Hz.
- Thuật toán tiến hành **Downsampling bằng cách lấy mẫu xen kẽ** (`df.iloc[:, :2, :]`) để giảm tần số xuống **100Hz**. Việc này giúp giảm một nửa khối lượng tính toán mà vẫn giữ nguyên được các đặc trưng tần số quan trọng của hành vi con người.
- **Chuẩn hoá (Normalization):**
 - Gia tốc chia 8 (kẹp $[-8, 8]$ g) để đưa về dải $[-1, 1]$.
 - Gyro chia 500 (kẹp $[-500, 500]$ dps) đưa về dải $[-1, 1]$. Giới hạn 500 dps được chọn dựa trên phân tích bách phân vị (p99.9) của tập dữ liệu thay vì lấy mốc 2000 dps gây lãng phí độ phân giải khi lượng tử hoá INT8.
 - **Lưu ý Lịch sử (Bug ở v25):** Ở phiên bản v25, dữ liệu Gyro bị nhầm đơn vị là rad/s thay vì dps. Do $1 \text{ rad/s} \approx 57.3 \text{ dps}$, toàn bộ biên độ Gyro đã bị thu nhỏ đi 57 lần. Việc v25 tiếp tục lấy giá trị này chia cho 2000 khiến dữ liệu Gyro khi lượng tử hoá về INT8 (dải -128 đến 127) trở nên cực kỳ bé, gần như quy về 0. Hệ quả: Mô hình v25 “mù” Gyro, chỉ dựa vào gia tốc (Accel) để đánh giá. Việc mất đi 3 trục Gyro vô tình loại bỏ một lượng lớn tín hiệu nhiễu phức tạp, giúp mạng Neural **học và hội tụ dễ dàng hơn rất nhiều** trong quá trình train. Tuy nhiên, cái giá phải trả là mô hình mất đi hoàn toàn khả năng cảm nhận không gian xoay, làm giảm hẳn sự nhạy bén thực tế (Recall) đối với những hành động phức tạp.

1.1.2. 1.2. Kỹ thuật Cắt cửa sổ (Windowing) & Gán nhãn nâng cao

Thay vì cắt Sliding Window mù quáng, hệ thống áp dụng kỹ thuật **Cắt theo sự kiện (Event-based Windowing)** cực kỳ thông minh: - **Kích thước cửa sổ:** 200 mẫu (tương đương 2 giây ở tần số 100Hz). - **Với nhóm hành động chuyển tiếp (Trans):** Hệ thống dùng hàm rolling-RMS của Gyro với ngưỡng kích hoạt $> 20 \text{ dps}$. Các vùng liên tục được gom thành 1 sự kiện và cắt duy nhất 1 cửa sổ 2 giây căn đúng vào đỉnh của sự kiện đó. Kỹ thuật này giúp mô hình chống nhiễu (robust) cực tốt kể cả khi hành động có nhiều đỉnh phụ (sub-peaks). - **Phân tách nhãn thông minh:** Những đoạn dữ liệu nằm xa sự kiện Gyro được gán nhãn Idle. - **Xử lý nhiễu đặc biệt (D18 & D19):** Các mẫu D18 (vấp khi đi) và D19 (nhảy nhót) chứa rất nhiều nhiễu. Thuật toán chủ động chỉ trích xuất đúng 1 cửa sổ Trans tại đỉnh gia tốc (SVM) cao nhất, toàn bộ phần đi bộ xung quanh bị vứt bỏ để tránh làm “bẩn” nhãn Walk hay Idle. - **Sliding Window truyền thống:** Chỉ áp dụng cho các hành động mang tính chu kỳ liên tục như Walk (D01, D02, D05) và Run (D03, D04).

1.1.3. 1.3. Data Augmentation

- Dữ liệu thuộc lớp Trans được làm phong phú (Augment) trong quá trình huấn luyện bằng cách **nhân biên độ tỷ lệ $\times 0.9$ và $\times 1.1$** thay vì dịch chuyển cửa sổ (sliding shift). Điều này giúp mô hình học được sự biến thiên lực của các đối tượng (người lớn tuổi/người trẻ) có vóc dáng khác nhau.

1.1.4. 1.4. Cấu trúc Tensor Đầu Vào

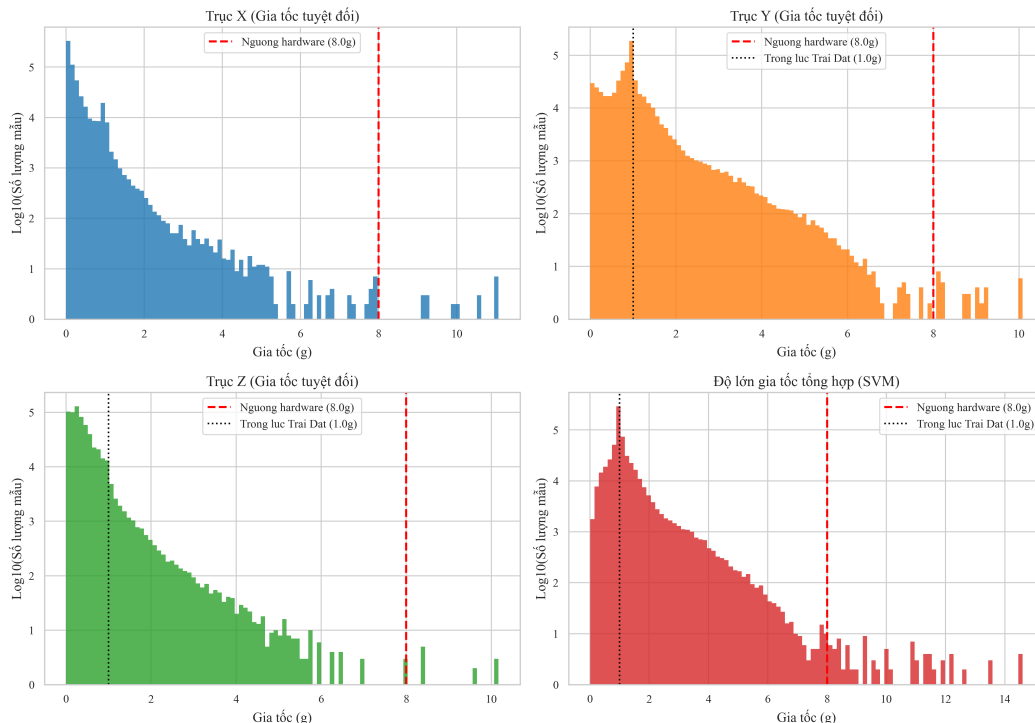
- Từ **v30** đến **v30_tcn_optimize_v2 (6 trục)**: $[1, 200, 6]$ ($a_x, a_y, a_z, g_x, g_y, g_z$).
- Đặc biệt với v31 (4 trục)**: Rút gọn đầu vào thành a_x, a_y, a_z và **Gyro RMS**. Gyro RMS được tính bằng $\text{Gyro}_{\text{RMS}} = \sqrt{g_x^2 + g_y^2 + g_z^2}$. Tensor đầu vào là $[1, 200, 4]$. Trên Firmware ESP32, vi điều khiển sẽ tự tính toán Gyro_{RMS} realtime từ 6 trục UART.

1.1.5. 1.5. Biểu đồ Phân bố Dữ liệu (Data Distribution & Thresholds)

Dưới đây là các biểu đồ Histogram (Log Base 10 Scale) minh họa trực quan toàn bộ phân bố của các trục tín hiệu gốc kèm vạch đỏ đánh dấu ngưỡng cắt (Threshold) tối ưu và vạch trọng lực Trái Đất (Gravity):

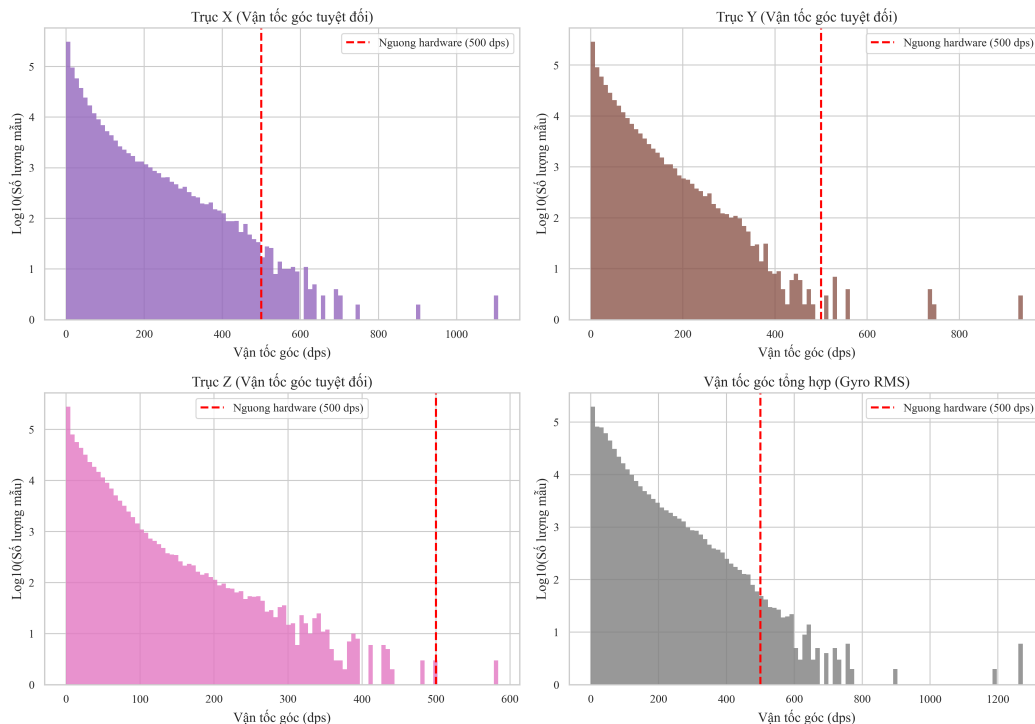
1. Phân bố Gia tốc (Accelerometer Distribution 2x2) (Layout 2x2 hiển thị phân bố các trục X, Y, Z và SVM tổng hợp. Vạch đỏ thể hiện ngưỡng cắt tối ưu 8.0g, vạch đen đứt thể hiện trọng lực Trái Đất 1.0g)

Phân Bố Giá Trị Tuyệt Đối Của Gia Tốc (Accelerometer Distribution)



2. Phân bố Vận tốc góc (Gyroscope Distribution 2x2) (Layout 2x2 hiển thị phân bố các trục X, Y, Z và RMS tổng hợp. Vạch đỏ thể hiện ngưỡng cắt tối ưu 500 dps)

Phân Bố Giá Trị Tuyệt Đối Của Vận Tốc Góc (Gyroscope Distribution)



Bảng Phân Tích Bách Phân Vị (Percentiles - Tính theo giá trị tuyệt đối):

Bảng dưới đây thống kê các mốc phân bố tín hiệu nhằm làm cơ sở thiết lập thang đo phân cứng (8g và 500 dps) để lượng tử hóa (Quantization INT8):

Trục Tín Hiệu	P50 (Trung vị)	P95	P99.9
Accel X (g)	0.09	0.98	2.16
Accel Y (g)	0.90	1.44	4.49
Accel Z (g)	0.27	0.97	2.62
Accel SVM (g)	1.00	1.60	5.22
Gyro X (dps)	4.39	70.13	404.49
Gyro Y (dps)	4.58	65.12	240.12
Gyro Z (dps)	1.77	49.93	227.13
Gyro RMS (dps)	11.39	111.17	463.57

Nhận xét: - **Gia tốc (Accel):** Ở bách phân vị 99.9%, mức gia tốc lớn nhất (của trục Y và SVM) chỉ xoay quanh mốc 4.5g đến 5.2g. Việc chọn thang đo phân cứng $\pm 8g$ bao phủ hoàn toàn mọi dao động khốc liệt nhất của các cú ngã mà không bị cắt xén dữ liệu (clipping). - **Vận tốc góc (Gyro):** Gần 95% thời gian các hoạt động, tốc độ xoay người (RMS) chỉ nằm dưới 111 dps. Ngay cả ở mốc 99.9% (những pha lộn vòng hoặc vấp ngã mạnh nhất), tốc độ xoay cực đại cũng chỉ chạm 463.57 dps. Đây là **“bằng chứng thép”** cho thấy việc chọn thang đo ± 500 dps là chính xác tuyệt đối, tránh lãng phí độ phân giải khi nén xuống INT8 (so với mốc ± 2000 dps của các nghiên cứu cũ).

Bảng Phân Tích Bách Phân Vị Theo Từng Hành Động (Đã Cắt Window):

1. Bảng Phân Tích Bách Phân Vị Gia Tốc (Accel) Theo Từng Hành Động Phần 1: Trục X và Trục Y

Nhãn (Class)	X-P50	X-P95	X-P99.9	Y-P50	Y-P95	Y-P99.9
Walk	0.10	0.39	0.80	0.95	1.48	2.18
Run	0.20	0.94	2.17	0.83	3.04	5.77
Idle_Lie	0.82	1.00	1.18	0.20	0.70	0.97
Idle_StandSit	0.04	0.22	0.84	0.97	1.01	1.37
Trans	0.07	0.95	1.29	0.85	1.05	3.27
Fall	0.25	1.31	5.20	0.45	1.36	5.79

Phần 2: Trục Z và Gia tốc Tổng hợp (SVM)

Nhãn (Class)	Z-P50	Z-P95	Z-P99.9	SVM-P50	SVM-P95	SVM-P99.9
Walk	0.23	0.69	1.56	1.00	1.61	2.38
Run	0.27	1.36	4.64	1.03	3.34	6.07
Idle_Lie	0.45	0.99	1.18	1.01	1.07	1.36
Idle_StandSit	0.27	0.56	1.02	1.01	1.04	1.54
Trans	0.38	0.90	1.85	1.00	1.20	3.62
Fall	0.46	1.17	3.79	0.98	2.19	8.18

2. Bảng Phân Tích Bách Phân Vị Vận Tốc Góc (Gyro) Theo Từng Hành Động Phần 1: Trục X và Trục Y

Nhãn (Class)	X-P50	X-P95	X-P99.9	Y-P50	Y-P95	Y-P99.9
Walk	11.29	53.71	159.85	15.20	65.55	145.94
Run	45.29	231.63	512.57	34.97	139.65	280.94
Idle_Lie	2.01	27.16	87.45	4.33	28.56	74.34
Idle_StandSit	1.40	30.33	109.62	4.09	21.00	95.40
Trans	13.37	67.87	263.82	6.47	46.08	108.01
Fall	22.03	143.07	401.67	25.63	177.73	403.14

Phần 2: Trục Z và Vận tốc góc Tổng hợp (RMS)

Nhãn (Class)	Z-P50	Z-P95	Z-P99.9	RMS-P50	RMS-P95	RMS-P99.9
Walk	11.60	43.21	80.08	30.59	87.21	177.25
Run	29.42	90.76	203.80	83.98	267.63	532.29
Idle_Lie	0.98	22.46	98.69	5.51	48.35	123.11

Nhãn (Class)	Z-P50	Z-P95	Z-P99.9	RMS-P50	RMS-P95	RMS-P99.9
Idle_StandSit	0.37	19.29	77.03	4.52	50.00	123.47
Trans	3.97	48.83	113.68	25.29	87.89	272.26
Fall	12.02	113.71	351.99	55.40	246.43	562.54

Nhận xét bổ sung: - **Về ngưỡng cắt 250 dps vs 500 dps:** Bảng trên cho thấy rõ ở mức cực đại (P99.9), tốc độ xoay của té ngã (Fall) có thể lên tới **562 dps**, và đỉnh điểm của hành động chạy (Run) là **532 dps**. Nếu chọn thang đo 250 dps, mô hình sẽ hoàn toàn “mù” (bị bão hòa tín hiệu) trước mọi sự kiện có tốc độ cao, khiến nó nhầm lẫn giữa một cú ngã mạnh và một bước chạy nhanh. - **Tách biệt các nhóm hành động tĩnh:** Nhóm Idle_Lie và Idle_StandSit có cường độ gia tốc và vận tốc góc cực kỳ thấp (P95 chỉ quanh quẩn 1g và 50 dps). Việc duy trì độ phân giải ở 500 dps (thay vì 2000) giúp mô hình TCN và ResNet bắt được những vi chấn động nhỏ này mà không bị làm tròn về 0 trong chuẩn INT8.

1.2. 2. PHÂN TÍCH CHI TIẾT CÁC MÔ HÌNH ĐÃ HUẤN LUYỆN

1.2.1. 2.1. Nhóm mô hình CNN Thuần (Ưu tiên ESP-NN)

Đây là nhóm mô hình hoàn toàn tuân thủ các quy tắc phần cứng của vi điều khiển ESP32-S3 (ESP-NN), chỉ sử dụng các toán tử được tăng tốc phần cứng như Conv2D, DepthwiseConv2D, MaxPool, Mean, và ReLU/ReLU6.

Tên mô hình	Cấu trúc & Kỹ thuật	Kích thước INT8	Tốc độ Firmware (SRAM)	Fall Recall (INT8)
v30	CNN thuần 6 trục. Sử dụng Separable Conv1D. Lược bỏ hoàn toàn Sigmoid và SE Block.	~25.0 KB	19.78 ms	98.00%
v31	CNN thuần 4 trục (Accel + Gyro RMS). Giống v30 nhưng đầu vào nhỏ hơn. Nhẹ nhất.	~24.9 KB	19.25 ms	98.00%

Đánh giá: CNN thuần là lựa chọn tối ưu tuyệt đối cho phần cứng. Tốc độ thực thi chớp nhoáng (< 20ms) cho phép triển khai Real-time với tần số quét cực cao mà không lo quá tải CPU.

1.2.2. 2.2. Nhóm mô hình Lai (CNN + LSTM)

Tên mô hình	Cấu trúc & Kỹ thuật	Kích thước INT8	Tốc độ Firmware (SRAM)	Fall Recall (INT8)
v30_lstm32	Trích xuất đặc trưng bằng CNN, sau đó đưa qua 1 lớp LSTM (32 units). Phải thiết lập Keras 2 Legacy để ép TFLite gom thành op UNIDIRECTIONAL_SEQUENCE_LSTM.	~39.2 KB	210.95 ms	94.50%

Đánh giá: Vì ESP-NN không hỗ trợ tăng tốc bằng vector phân cứng cho LSTM, vi điều khiển phải chạy bằng code C++ thuần. Tốc độ giảm gấp 10 lần so với CNN.

1.2.3. 2.3. Nhóm mô hình ResNet1D

Tên mô hình	Cấu trúc & Kỹ thuật	Kích thước INT8	Tốc độ Firmware (SRAM)	Fall Recall (INT8)
v30_resnet1d	ResNet1D tích hợp Squeeze-and-Excitation (SE) Block.	~83.3 KB	45.75 ms	96.00%

Đánh giá: Mặc dù là CNN, nhưng việc tồn tại hàm Sigmoid (trong khối SE Block) và GlobalMaxPooling (REDUCE_MAX) khiến một phần mạng không được tăng tốc. Ngoài ra, việc chứa nhiều tensor nhỏ rải rác khiến lượng Metadata (thông điệp đi kèm) phình to (file tflite nặng tới 83KB dù số lượng tham số ít).

Giải mã sự suy giảm tốc độ của v30 so với v25 (45ms vs 32ms): Dù v30_resnet1d giữ nguyên kiến trúc gốc giống hệt v25 (cùng version Keras, cùng thuật toán nén INT8), tốc độ suy luận thực tế trên ESP32 vẫn bị chậm đi (từ 32ms lên 45.75ms). Khi “mổ xẻ” các tensor trong file .tflite, nguyên nhân cốt lõi đã lộ diện ở tầng lượng tử hóa (Quantization): - Việc v30 phải học trên tập dữ liệu Trans khó hơn (có augmentation) khiến mạng phải thay đổi mạnh phân bố trọng số để hội tụ. Hệ quả là các dải Lượng tử hóa (scale và zero_point) của các lớp Conv ở v30 bị thay đổi hoàn toàn so với v25 (Ví dụ: out_scale của lớp Conv2D số 2 bị lệch từ 0.1511 ở v25 xuống 0.1257 ở v30). - Thư viện tăng tốc ESP-NN cực kỳ nhạy cảm với dải scale. Nó cần tính toán một biến $quantized_multiplier = (in_scale * filter_scale) / out_scale$. Nếu tỷ lệ này vượt ra khỏi giới hạn các lệnh dịch bit (bit-shift) tối ưu của vi điều khiển, ESP-NN sẽ tự động ngắt tính năng Hardware Acceleration và đẩy lớp Conv đó về chạy bằng vòng lặp C++ thuần (Reference Kernel) nhằm đảm bảo không bị tràn số (overflow). Chính sự thay đổi trọng số ngẫu nhiên do dữ liệu khó đã vô tình đẩy một vài lớp Conv của v30 rơi vào trường hợp “cắm” này, tạo ra độ trễ ~13ms.

1.2.4. 2.4. Nhóm mô hình TCN (Temporal Convolutional Network)

TCN sử dụng kiến trúc tích chập theo thời gian, thường dùng Dilated Convolutions (Tích chập giãn nở) để tăng trường nhìn (Receptive Field) mà không làm tăng tham số.

Tên mô hình	Cấu trúc & Kỹ thuật	Kích thước INT8	Tốc độ Firmware (SRAM)	Fall Recall (INT8)
v30_tcn	TCN Cổ điển. Sử dụng Dilated Convolutions (Dilation > 1).	~105.0 KB	2058.50 ms	100.0% (Mẫu)
v30_tcn_optimize	TCN thiết kế lại: Bỏ Dilation, thay bằng chuỗi DepthwiseConv2D và Conv2D chuẩn.	~66.5 KB	184.98 ms	96.50%
v30_tcn_optimize_v2	Tối ưu lại Stride và Kernel Size so với v1 để tìm điểm cân bằng.	~65.6 KB	217.75 ms	96.50%

Đánh giá: - **TCN Cổ điển** là “thảm họa” phần cứng: Mất hơn 2 giây để dự đoán 1 mẫu. Lý do là ESP-NN từ chối tăng tốc bất kỳ Convolution nào có `Dilation_rate > 1`. - **TCN Optimize** đã chứng minh việc thiết kế mạng thuận theo phần cứng (Hardware-aware Neural Architecture Design) là bắt buộc. Việc đổi sang Conv chuẩn giúp tốc độ tăng hơn 10 lần (184ms vs 2058ms).

1.3. 3. KẾT LUẬN & ĐỀ XUẤT CHO TRIỂN KHAI THỰC TẾ

1. **Hiệu suất (Inference Speed):** ESP32-S3 chỉ toả sáng khi mạng **tuân thủ nghiêm ngặt chuẩn ESP-NN** (Chỉ dùng Conv chuẩn, Depthwise, MaxPool, Mean, ReLU). Bất kỳ toán tử ngoại lai nào (LSTM, Sigmoid, Dilated Conv, Reduce_Max) đều sẽ kéo tốc độ tụt lùi từ 3 đến 100 lần.
2. **Kích thước Flash/RAM:** Mô hình có thiết kế ít khối (Layer) nhưng kích thước lớn sẽ tiết kiệm bộ nhớ hơn là mô hình ít tham số nhưng xé lẻ thành hàng chục khối nhỏ (như SE Block của ResNet1D), do overhead từ FlatBuffer Metadata của TFLite.
3. **Đề xuất Deploy:** Phiên bản **v31 (CNN 4 trục)** và **v30 (CNN 6 trục)** là 2 ứng cử viên sáng giá nhất cho sản phẩm cuối cùng. Chúng đạt độ trễ tuyệt hảo (~19ms), dung lượng siêu gọn (~25KB) mà vẫn giữ vững tỷ lệ bắt té ngã cực kỳ ấn tượng (98%). Sự đánh đổi (Trade-off) bằng các mô hình phức tạp hơn như TCN hay LSTM là không mang lại đủ lợi ích so với lượng độ trễ phát sinh.